

TxKernel 决赛线上阶段测例启动链路修复与运行环境优化报告

一、背景与目标

本次工作聚焦于 TxKernel 在 RV64 SMP4 配置下运行决赛测例时暴露出的用户态启动链路问题。目标不是单独优化某个 syscall 的微观性能，而是补齐内核向用户态暴露的 Linux 运行环境行为，使初赛与决赛硬盘镜像都能正确挂载并运行，修复 AI

对应核心提交为：

2a01b073 oscomp: repair ext4 and non-ltp runtime

该提交之后，focused SMP4 witness 中 basic-musl, busybox-musl 的得分从 29/157 提升到 155/157，说明原先主要瓶颈并非测例自身逻辑，而是内核启动、文件系统布局、动态链接器路径与 VFS syscall 语义共同组成的运行环境不完整。

二、问题定位与根因分析

最初现象是：SMP4 下 OSComp non-LTP 测例无法顺利跑完，尤其是 basic-musl 与 busybox-musl 得分极低。进一步分析发现，失败不是单个业务 syscall 返回值错误，而是用户态程序启动前后的系统环境与 Linux 预期不一致。

根因可以分为四类。

第一，OSComp 镜像布局不稳定。不同测试镜像可能将 musl 用户态放在 /musl，也可能放在 /musl/musl。旧启动逻辑对路径有固定假设，导致 busybox、脚本和 libc 查找不稳定。

第二，测例需要可写工作目录。basic-musl 中的 mkdir_、unlink、openat、mount、umount 等用例会创建、删除或修改文件。如果直接在只读 ext4 sdcard 挂载点上运行，会把文件系统只读性误判成 syscall 失败。

第三，动态 ELF loader 兼容路径缺失。musl/glibc 风格动态程序会依赖 /lib/ld-musl-riscv64.so.1 或 /lib/ld-linux-riscv64-lp64d.so.1 等传统 Linux 路径。原 rootfs shim 没有稳定发布这些入口，导致部分动态程序在 execve 阶段失败。

第四，VFS/syscall 语义与 Linux 有偏差。BusyBox mkdir -p 可能调用 mkdirat("/usr/", ...)，Linux 接受创建目录路径末尾的 /；同时 statx/newfstatat 应通过调用进程的 mount namespace 解析路径。旧实现对这些边界处理不足，使测例看到的文件系统状态与预期不一致。

因此，本问题的本质是：TxKernel 已经具备运行用户态程序的基础能力，但 OSComp 测例要求的是一个更完整、更 Linux-like 的启动环境契约。只修某个失败用例无法根治问题，必须从启动编排、rootfs shim、动态 loader、VFS syscall 语义四层同时补齐。

三、修复设计与实现

设计将修复划分为四个层次。

1. 启动路径自适应

内核启动 OSComp sdcard 测例时，不再固定假设 /musl/musl 一定存在，而是在启动命令中根据实际目录结构选择运行根。

```
fn build_oscomp_sdcard_cmd<P: tx_hal::TxPlatform>() -> alloc::string::String {
    use alloc::string::String;

    let mut cmd = String::from(
        "tx_musl_root=/musl/musl; \
        [ -d \"$tx_musl_root/basic\" ] || tx_musl_root=/musl; \
```

```

        tx_bb=\"${tx_musl_root}/busybox\"; \
        cd \"${tx_musl_root}\",
    );

    // 后续按 OSCOMP_GROUPS 选择 basic-musl、busybox-musl 等测例组
}

```

这段逻辑解决了 flat 布局与 nested 布局不一致的问题，使同一套内核启动逻辑可以适配不同 OSComp 镜像。

2. basic-musl 使用 tmpfs 工作区

basic-musl 测例会写文件，因此不能直接在只读 sdcard 上执行。修复后，内核启动脚本将官方测例复制到 /tmp/tx-basic-musl，再从该可写目录运行。

```

fn append_basic_script(cmd: &mut alloc::string::String) {
    cmd.push_str(
        "; ./busybox echo \"#### OS COMP TEST GROUP START basic-musl ####\"\\
        ; tx_basic_work=/tmp/tx-basic-musl; ./busybox rm -rf \"${tx_basic_work}\"\\
        ; ./busybox mkdir -p \"${tx_basic_work}\"\\
        ; ./busybox cp -r \"${tx_musl_root}/basic/.\" \"${tx_basic_work}\"\\
        ; (cd \"${tx_basic_work}\"; for tx_basic_case in brk chdir clone close dup2
dup execve exit fork fstat getcwd getdents getpid getppid gettimeofday mkdir_ mmap
mount munmap openat open pipe read sleep times umount uname unlink wait waitpid write
yield; do \"${tx_bb}\" echo \"Testing $tx_basic_case :\"; \"${tx_musl_root}/lib/libc.so\"
\"./${tx_basic_case}\"; done)\\
        ; ./busybox echo \"#### OS COMP TEST GROUP END basic-musl ####\"\",
    );
}

```

设计重点是：测例输入仍来自官方 sdcard，保证测试内容不被篡改；运行时写入发生在 tmpfs，避免只读 ext4 挂载误伤可写用例。

3. rootfs 发布 loader 与 shebang shim

为了让动态 ELF 和 shell 脚本能按 Linux 常见路径运行，test-init 阶段补充 /bin、/usr/bin/env 和 loader 兼容入口。

```

if [ ! -e /lib/ld-musl-riscv64.so.1 ]; then
    if [ -e /musl/lib/libc.so ]; then
        link_force /musl/lib/libc.so /lib/ld-musl-riscv64.so.1
    elif [ -e /musl/musl/lib/libc.so ]; then
        link_force /musl/musl/lib/libc.so /lib/ld-musl-riscv64.so.1
    fi
fi

if [ ! -e /lib/ld-linux-riscv64-lp64d.so.1 ]; then
    if [ -e /musl/lib/libc.so ]; then
        link_force /musl/lib/libc.so /lib/ld-linux-riscv64-lp64d.so.1
    elif [ -e /musl/musl/lib/libc.so ]; then
        link_force /musl/musl/lib/libc.so /lib/ld-linux-riscv64-lp64d.so.1
    fi
fi

```

这部分修复把“程序能不能被 execve 启动”从偶然路径匹配，变成明确的 rootfs 兼容启动。

4. 修正 VFS syscall 语义

mkdirat 对路径末尾的/做 Linux 兼容归一化，不影响 unlink/link 等更严格的路径操作。

```
fn split_mkdir_path(path: &[u8]) -> (&[u8], &[u8]) {
    let mut end = path.len();
    while end > 1 && path[end - 1] == b'/' {
        end -= 1;
    }
    split_path(&path[..end])
}
```

newfstatat 则根据 dirfd、AT_EMPTY_PATH 和调用进程上下文选择正确解析起始路径，再从该进程视角解析路径。

```
pub(super) async fn sys_newfstatat<'a>(
    args: [u64; 6],
    ctx: &SyscallCtx<'a>,
) -> SyscallResult {
    let dirfd = args[0] as i32;
    let path_uaddr = args[1];
    let statbuf_uaddr = args[2];
    let flags = args[3] as u32;

    if statbuf_uaddr == 0 {
        return SyscallResult::Error(EFAULT_VALUE);
    }

    let path = match read_user_cstr(&ctx.aspace, path_uaddr, EXECVE_PATH_MAX) {
        Ok(p) => p,
        Err(ReadCStrError::TooLong) => return
SyscallResult::Error(ENAMETOOLONG_VALUE),
        Err(ReadCStrError::OutOfMemory) => return SyscallResult::Error(ENOMEM_VALUE),
        Err(ReadCStrError::Fault(_)) => return SyscallResult::Error(EFAULT_VALUE),
    };

    // 修改
    let cwd = if path.is_empty() && (flags & AT_EMPTY_PATH != 0) {
        match ctx.process.cwd() {
            Some(d) => d,
            None => return SyscallResult::Error(ENOENT_VALUE),
        }
    } else {
        match dirfd_anchor_for_path(dirfd, &path, ctx) {
            Ok(cwd) => cwd,
            Err(result) => return result,
        }
    };

    // 解析
}
```

这类修复的意义在于：OSComp 测例不只是调用 syscall，还隐含依赖 Linux 路径解析规则。只有把这些规则放回 syscall/VFS 边界，测试才能稳定运行。

四、实验分析

本次主要指标是 OSComp focused witness 的通过数和通过率。由于该测例重点是运行环境正确性，不是编译型 workload，因此不把“内核编译时间”作为主指标。

测试项	修改前	修改后	变化
basic-musl,busybox-musl 总分	29/157	155/157	+126
basic-musl	0/102	100/102	+100
busybox-musl	29/55	55/55	+26
总通过率	18.47%	98.73%	+80.26 个百分点
按通过用例数计算的提升倍数	baseline	5.34x	155 / 29

后续还进行了 libcbench-musl SMP 对照实验：

配置	27 条 time: 汇总	相对 SMP1
SMP1 default	5.877901 s	baseline
SMP4 default	6.091749 s	0.9649x
SMP4 child-spread oracle	6.039818 s	0.9732x

该结果说明：本次修复显著提升了测例可运行性和正确性，但没有声称 SMP4 性能已经超过 SMP1。SMP4 性能优化仍需要后续 PELT/调度策略与 per-case owner counters 支撑。

五、可复现步骤

审核者可以按如下步骤复现本设计。注意本修复位于 ext4-main 分支上，不在 main 分支上。

```
cargo test -p tx-kernel init::exec::tests -- --nocapture
cargo test -p tx-ext4-format --lib -- --nocapture
```

构建 OSComp RV64 镜像与提交包：

```
make oscomp-build-rv64 oscomp-submit-rv64
```

运行 focused SMP4 witness：

```
make oscomp-qemu-rv64-smp4 \
  OSCOMP_GROUPS=basic-musl,busybox-musl \
  OSCOMP_TEST_INIT_CMDLINE='init=/tx-test-init tx.test_init=1 tx.oscomp.observe=0
tx.oscomp.observe_dump=0' \
  OSCOMP_OUT_RV_SMP4=target/oscomp/fix-nonltp/os_serial_out_rv_smp4-basic-busybox.txt
```

评分：

```
python3 tools/oscomp-judge.py \
  target/oscomp/fix-nonltp/os_serial_out_rv_smp4-basic-busybox.txt \
  target/oscomp/testdata
```

期望结果是 basic-musl 接近 100/102，busybox-musl 为 55/55，合计约 155/157。

六、AI 使用说明

本次开发过程中使用 AI 辅助完成以下工作：

- 从 OSComp 失败现象中归纳共性根因，区分“测例脚本问题”和“内核运行环境契约问题”。
- 定位相关代码路径，包括启动编排、rootfs shim、动态 loader 兼容入口、VFS syscall 语义修复。
- 整理实验数据、复现命令和报告结构。

所有结论均以本地 git commit、代码 diff、测试命令和 OSComp judge 分数为依据。未将尚未闭合 receipt 的 rustc clean-build 诊断线描述为已完成性能优化；性能相关结论只使用已有的 focused witness 与 libcbench-musl 对照数据。

七、总结

本次修复的核心价值是将 TxKernel 的 OSComp 用户态运行环境从“能启动部分程序”推进到“能满足 basic/busybox 测例所依赖的 Linux 文件系统与动态执行契约”。通过路径自适应、tmpfs 工作区、loader shim、VFS syscall 语义修正，SMP4 下 focused non-LTP 测例从 29/157 提升到 155/157。这说明本次工作有效消除了测例启动链路中的主要阻塞点，为后续完整 non-LTP、LTP 和真实性能优化打下了基础。